

SIMAT'S ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – SHORT ANSWER TEST

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 3 – Short Answer Test
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problems.		
Student Name:	K. Karthik	Reg. No.:	192525040
Section / Branch:	AI & ML	Date:	04/09/2026

Code of Conduct

I, K. Karthik (192525040) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: karthik

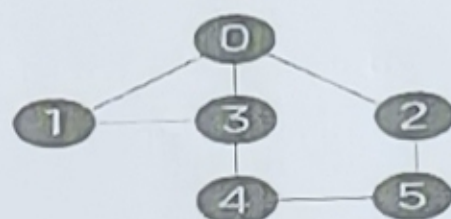
Instructions

- This test contains 10 short answer test questions. Total: 100 Marks.
- When a student answers using an Analytical problem-solving, in evaluation the answer should be with the following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A – Short Answer Test

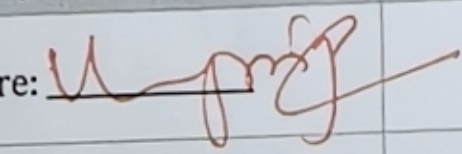
1. Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations: 1) Invert the elements in the stack, 2) Pop (), 3) Pop (), 3) Pop Construct AVL tree for the following elements. Mention the
2. Given a collection of elements 10,20,30,40,50,60,70,80. The Key element to be search 70. Compare how searching is performed in Linear Search and Binary Search. Identify the efficient search method. Justify.
3. What is the postfix and prefix forms of the following expression? $A+B*(C-D)/(P-R)$
4. Construct AVL tree for the following elements. Mention the Rotations in each step of insertion. 1,2,3,4,5,6,7,8,9,10
5. Construct a heap by inserting 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45 into a binary heap.
6. Construct a B-Tree of Order 3 by inserting numbers 34,26,45,12,87,56,78,22,23,24,65,85,95.
7. Perform the merge sort for the following inputs. Also, show the result for each step of iteration. 40,20,70,14,60,61,97,30.
8. How the following data set can be sorted by quick sort. Show each step-in sorting. 78,23,31,88,43,55,67,55.
9. Perform Open Hashing & Linear Probing for the given set of 0,1,81, 4,64,25,16,36, 9 and 49.
10. Consider the following graph & construct BFS.



Graph with six nodes

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates complete and accurate understanding of concepts	Good understanding with minor gaps	Basic understanding	Limited understanding
Accuracy of Answer	25	Answers are completely correct and relevant	Mostly correct with minor errors	Partially correct	Mostly incorrect
Problem-Solving & Reasoning	20	Provides logical reasoning and appropriate steps	Good reasoning with minor gaps	Basic reasoning	Lacks logical reasoning
Clarity & Relevance	15	Answers are precise, clear, concise, and directly address the question	Mostly clear and relevant	Somewhat unclear or incomplete	Irrelevant or unclear answers
Time Management & Completion	10	Completes all questions accurately within the allotted time	Completes most questions on time	Requires additional time	Unable to complete the test

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Stack operations

Given Stack: 22, 55, 33, 66, 88

Assume 88 is the Top before inversion

Top $\rightarrow$ 88

66

33

55

22

Operation: 1 Invert the stack

After reversing all elements

Top $\rightarrow$ 22

55

33

66

88

Operation: 2 pop()

Popped = 22

Stack: Top $\rightarrow$ 55

33

66

88

Operation: 3 pop()

Popped = 55

Stack: Top $\rightarrow$ 33

66

88

Operation: 4 pop()

Popped = 33

Final Stack: Top $\rightarrow$ 66

88

Final answer

→ Inverted stack = 22, 55, 33, 66, 88

→ First pop = 22

→ Second pop = 55

→ Third pop = 33

→ Remaining stack = 66, 88

The original question specifies the five elements, stack and the pop operations.

② Linear search vs Binary Search

Given : 10, 20, 30, 40, 50, 60, 70, 80

Search key : 70

The data is already sorted, so both searches can be demonstrated

Linear search : Linear search checks elements one by one

<u>Step</u>	<u>Elements</u>	<u>Comparison</u>
1	10	$10 \neq 70$
2	20	$20 \neq 70$
3	30	$30 \neq 70$
4	40	$40 \neq 70$
5	50	$50 \neq 70$
6	60	$60 \neq 70$
7	70	$70 = 70$ Found

Therefore : 7 comparisons

Linear search complexity

→ Best case : $O(1)$

→ Average case : $O(n)$

→ Worst case : $O(n)$

Binary Search :: Array:

10	20	30	40	50	60	70	80
----	----	----	----	----	----	----	----

Step 1 :: Low = 0 : High = 7

$$\text{mid} = (0 + 7) / 2 = 3$$

element at index 3 = 40

Since $70 > 40$ search right half

Step 2 :: Search

50	60	70	80
----	----	----	----

$$\text{mid} = 5 = 60$$

Since $70 > 60$ search right half

Step 3 ::

70	80
----	----

$$\text{mid} = 6$$

70 = 70

element found

Therefore :: 3 comparison

Binary Search complexity

-> Best case : $O(1)$

-> Avg case : $O(\log n)$

-> worst case : $O(\log n)$

Efficient method :: Binary search is more efficient for this example

Linear search = 7 comparisons

Binary search = 3 comparisons

Justification :: Binary search eliminates approximately half of the remaining search space after every comparison, it requires the data to be sorted.

③ Postfix and Prefix

Expression: $A + B * (C - D) / (P - R)$

Step 1: parentheses: $(C - D)$
 $(P - R)$

Step 2: multiplication: $B * (C - D)$

Step 3: Division $[B * (C - D)] / (P - R)$

Step 4: Addition $A + [B * (C - D)] / (P - R)$

convert operations after their operands:
 $ABCD - * PR - / +$

therefore postfix: $ABCD - * PR - / +$

Prefix: convert operations before their operands
 $+ A / + B - CD - PR$

Therefore

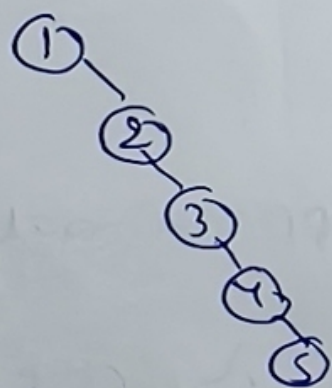
Prefix: $+ A / + B - CD - PR$

The expression specified in the paper is $A + B * (C - D) / (P - R)$

④ AVL Tree

Elements 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Insert

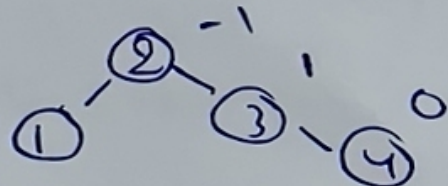


in balance

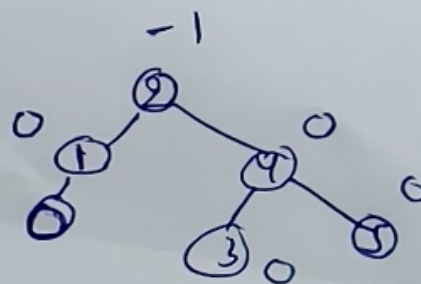
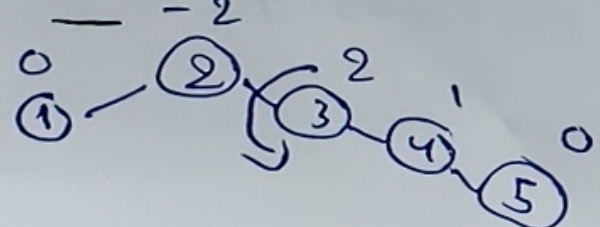
RR

imbalance ab 1

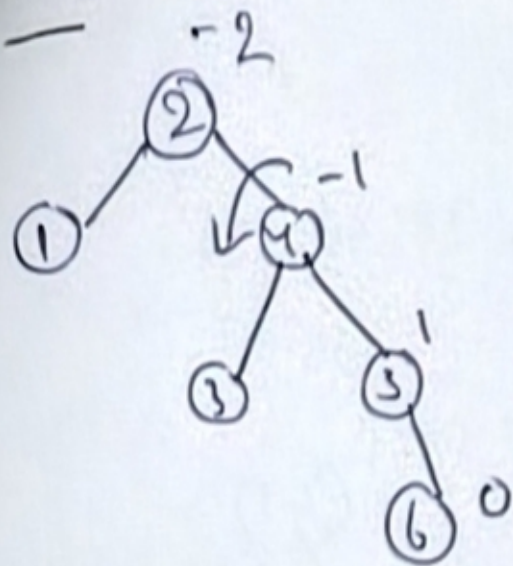
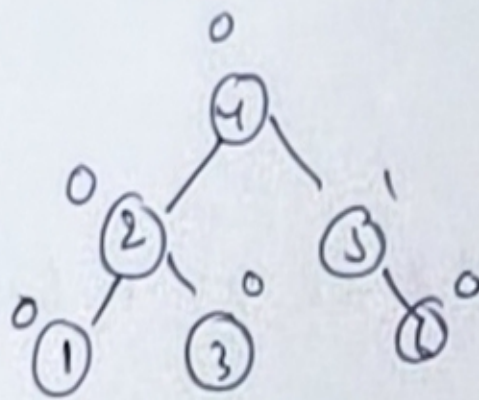
Then on RR



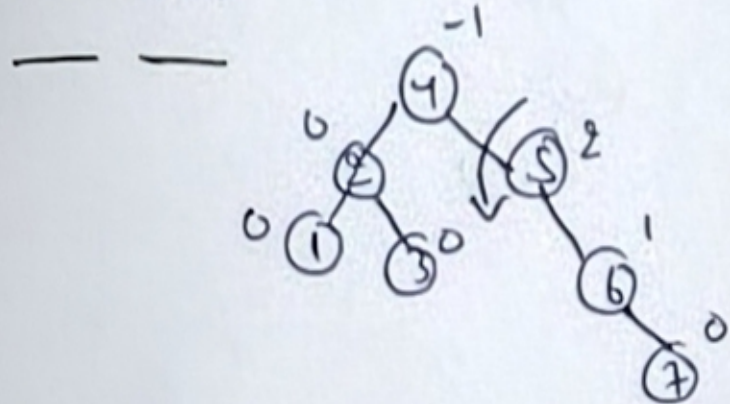
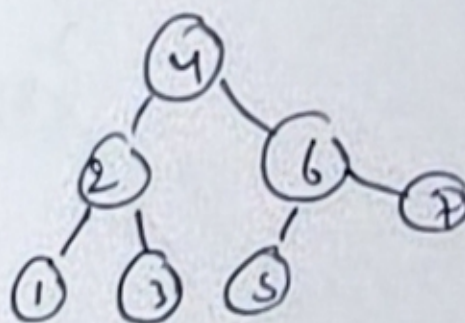
Insert 5



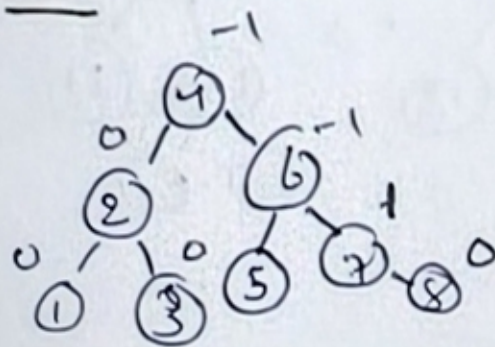
: 6

 $\Rightarrow$ 

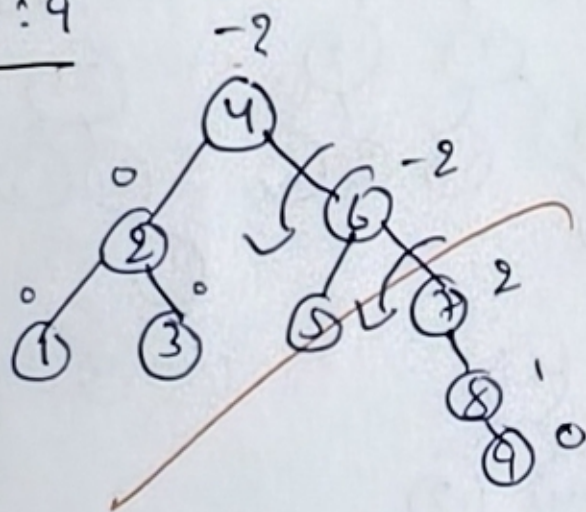
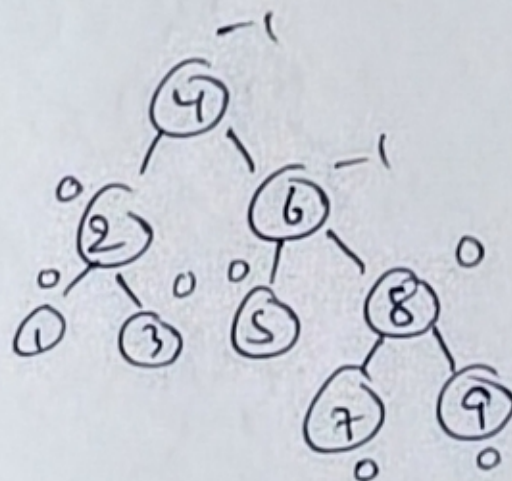
Insert: 7

 $\Rightarrow$ 

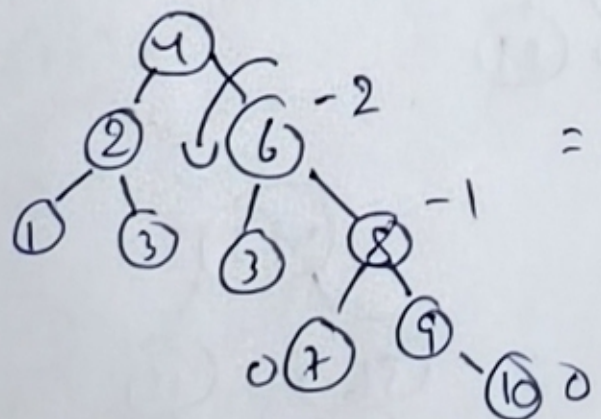
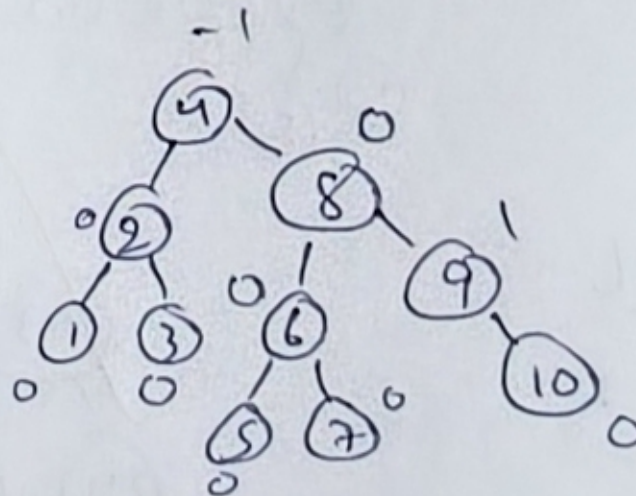
Insert 8:



Insert: 9

 $\Rightarrow$ 

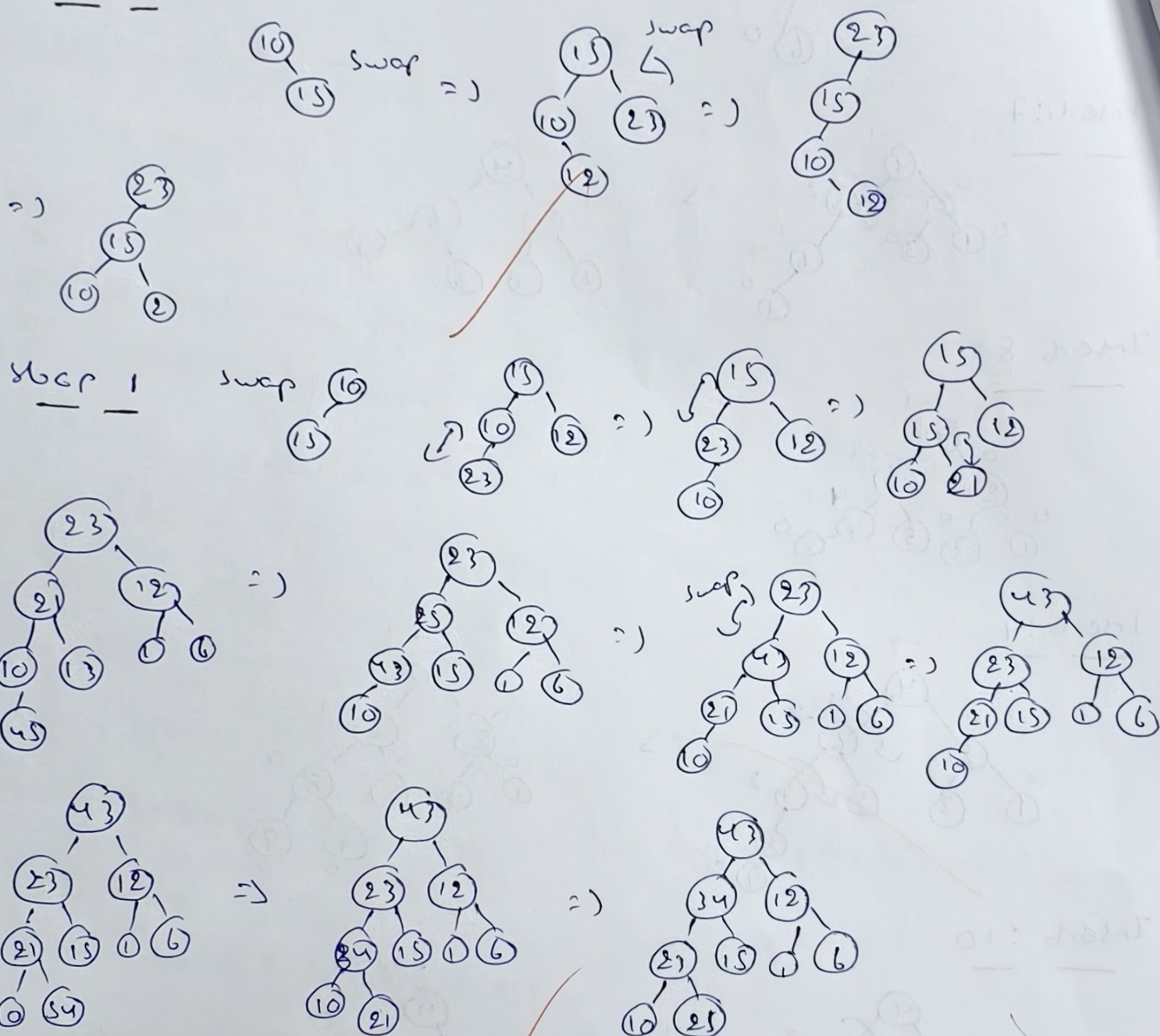
Insert: 10

 $\Rightarrow$ 

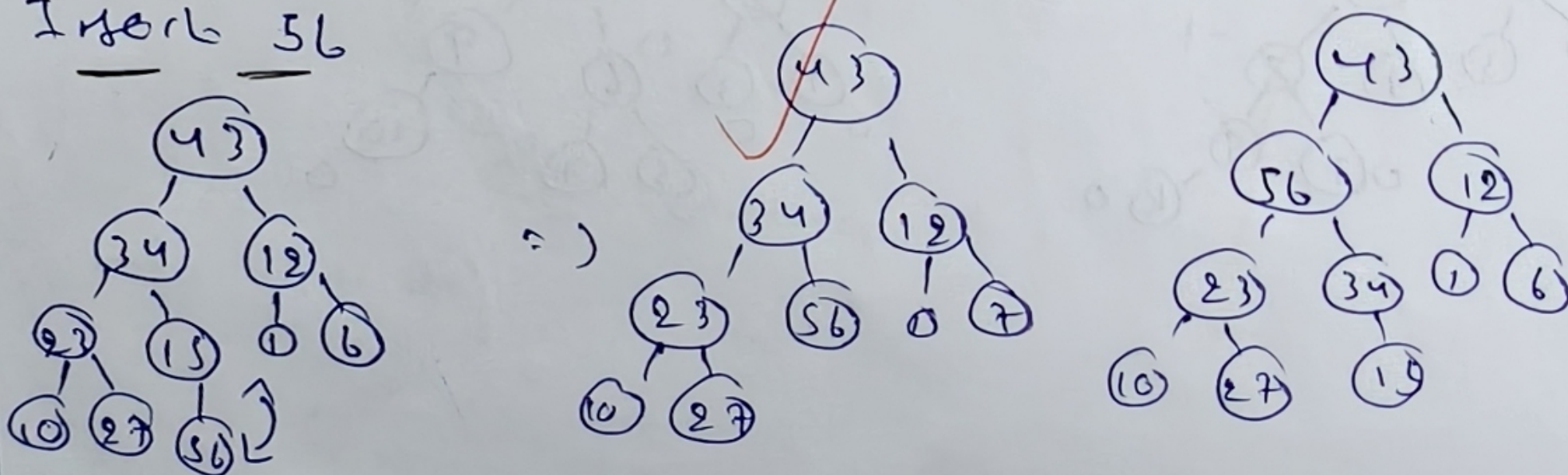
5 Binary Heap

elements: 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45

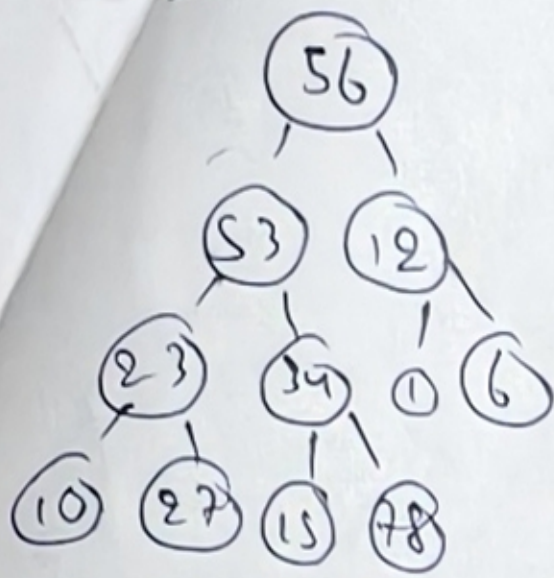
max heap:



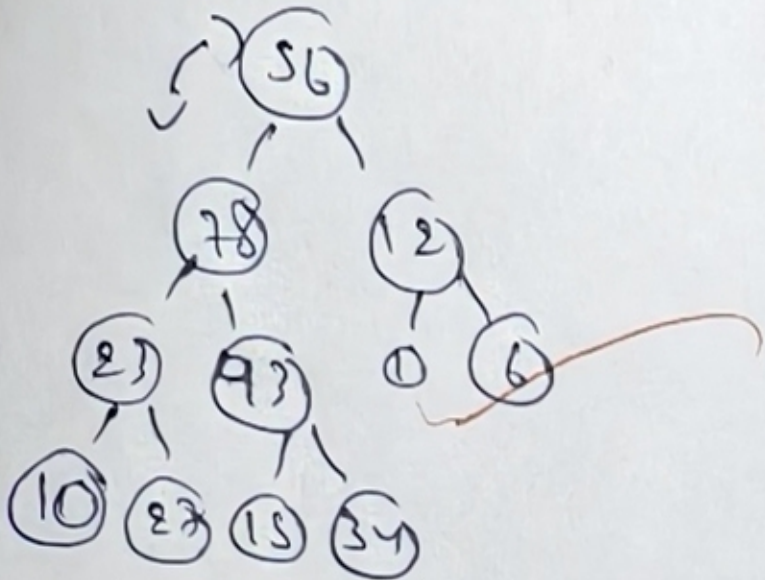
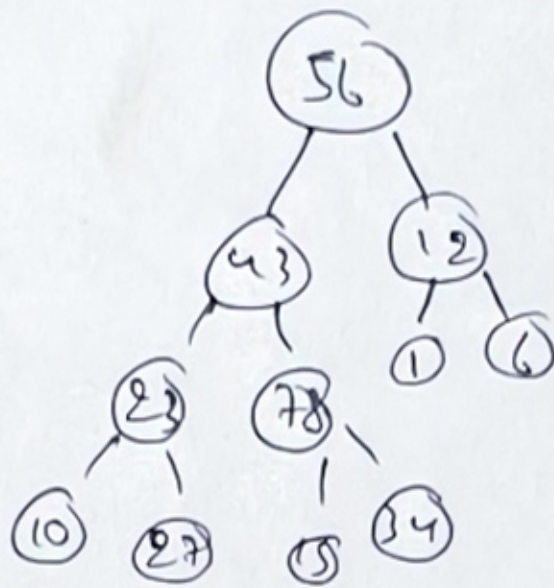
Insert 56



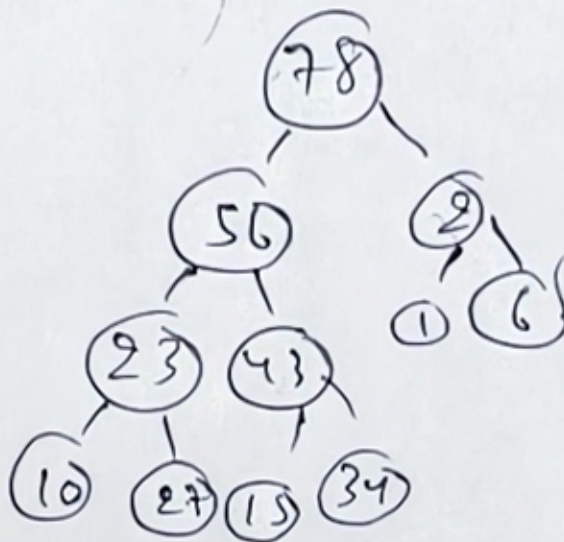
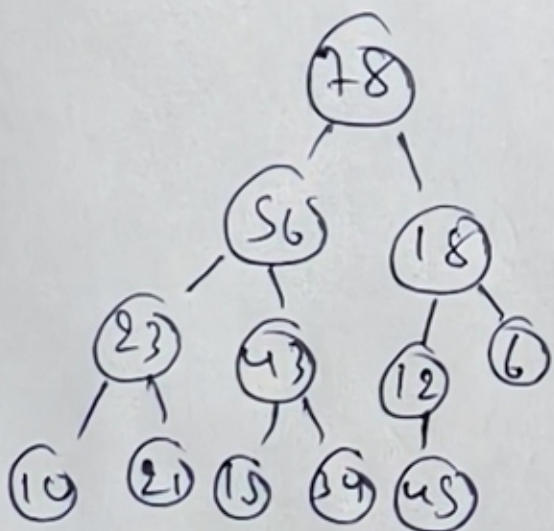
78



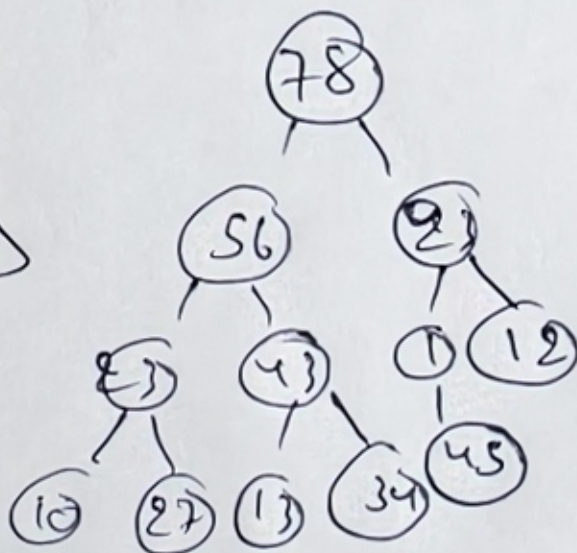
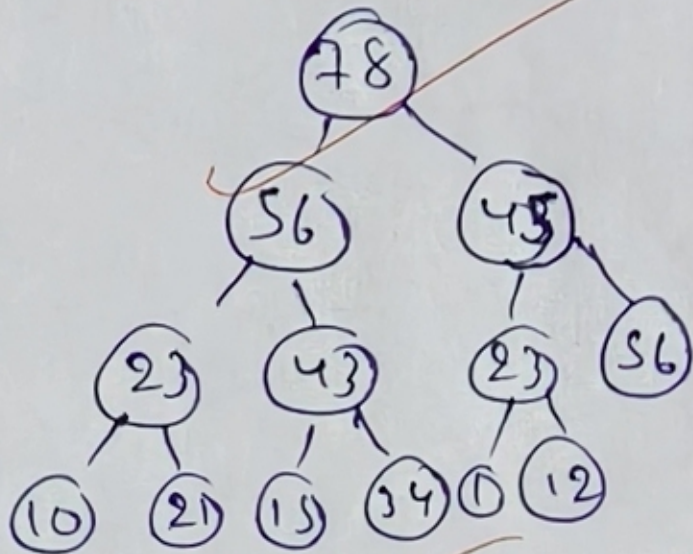
=>



=>

Insert: 23

=>

Insert: 43

u g o t